

“A Machine Learning-Enhanced Honeypot Security System for Dynamic Threat Detection”

A thesis report submitted to the department of Computer Science and Engineering of the World University of Bangladesh in partial fulfillment of the requirement for award of the degree of Bachelor of Science in Computer Science & Engineering

ABSTRACT

This research explores the integration of machine learning (ML) with honeypot systems to create a dynamic and adaptive security solution for cybersecurity. Honeypots serve as decoy systems designed to lure and capture cyber attackers for study and defense. The introduction outlines the growing challenges in cybersecurity, which are exacerbated by increasingly sophisticated attacks. It emphasizes the role of honeypots and proposes the enhancement of traditional systems through ML to detect new attack patterns in real time.

The research objectives include designing and implementing a honeypot system capable of attracting and logging malicious activity, classifying various types of cyberattacks, and evaluating the effectiveness of the ML-enhanced system. It also highlights the use of ML algorithms for analyzing honeypot data to identify emerging threat patterns.

The study justifies the integration of ML by addressing the limitations of traditional security systems, which often fail to detect novel threats. By combining honeypots with ML, the system enables adaptive, behavior-based threat detection, offering a proactive defense mechanism that can improve the detection of both known and unknown cyberattacks.

A review of existing systems reveals the limitations of traditional honeypot technologies, with a focus on both high- and low-interaction honeypots. This research proposes a system that balances safety, efficiency, and data richness, shifting from signature-based to ML-based detection approaches. The integration of ML enhances the system's ability to perform automated data analysis, reducing dependency on manual intervention.

The methodology for this research involves system design, data collection, and the integration of ML models (Logistic Regression, Random Forest, and XGBoost). The system detects new threats

by analyzing logs from the honeypot and performing data preprocessing and feature extraction. The system's performance is evaluated using various metrics, including accuracy, precision, recall, and F1-score.

The study's outcomes demonstrate that the system achieves 96% accuracy, precision, recall, and F1-score. It operates in real time, providing a lightweight and practical solution for research, training, and small to medium-sized businesses (SMBs). The study also highlights future directions, including expanding training with larger datasets and integrating deep learning models for more complex attack detection.

Introduction

According to data released by GData, the number of malware continues to increase. In the second half of 2014 there were 4,150,068 new types of malware. This number increased by almost 2.3 times from the first semester of the same year which amounted to 1,848,617 types of malwares. So that overall, the recorded malware in 2014 was 5,998,685. Based on the category, the highest number of malwares is in Trojan Horse malware while in Adware malware is the most significantly increased malware.

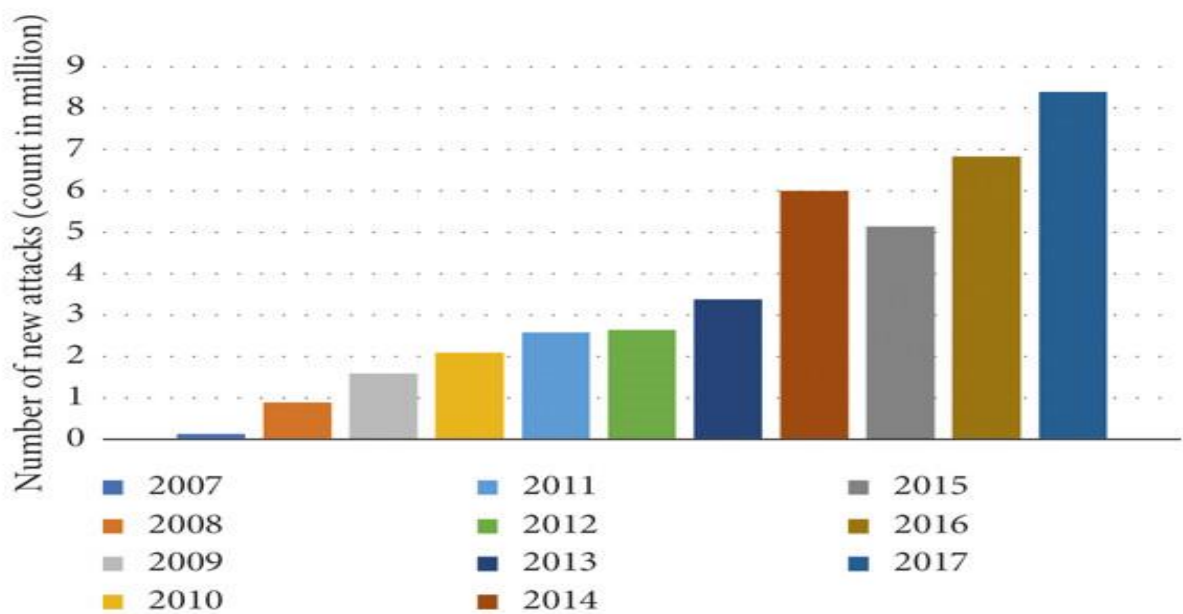


Fig: 1.1 GData New type of Attack Report

The rising incidence of high-sophistication cyberattacks renders conventional security systems unsuitable for end-to-end threat detection and countermeasures. Traditional Intrusion Detection Systems (IDSs), while expert in identifying patterns of attack, are likely to fail in handling emergent, rapidly evolving threats that outmaneuver static defense systems. It is in this context that honeypots have emerged as an essential component in proactive cybersecurity defense systems. By impersonating weak systems, honeypots lure attackers and capture valuable behavior data, enabling analysts to examine attacker strategy and motive.

However, conventional honeypot deployments are mostly reactive and rule- or human-based analysis. These restrictions reduce their ability to respond against new attack vectors in real time. The addition of Machine Learning (ML) into honeypot platforms is a disruptive potential. ML enables automated threat behavior classification and prediction using data-driven insight. This combination enhances the dynamic capabilities of honeypots, transforming them not only into traps but also intelligent agents in the cyber security space.

This study hypothesizes the development of a more advanced honeypot system using machine learning that is able to capture malicious activity, preprocess, and apply supervised learning models to identify and categorize multiple cyberattacks in real time. The system must provide high detection accuracy, low false positives, and adaptability to changing threats, offering a viable defense system for small- to medium-sized network configurations.

Objectives

The objectives of this research are:

- To design and implement a honeypot system that attracts and logs malicious activity.
- To apply machine learning algorithms for analyzing honeypot data and detecting threat patterns.
- To classify types of cyberattacks (e.g., port scans, brute force, malware injection).
- To evaluate the effectiveness and accuracy of the proposed system.

Methodology

The methodology adopted in this research follows a structured pipeline consisting of four major stages: System Design, Data Preprocessing, Machine Learning Integration, and Evaluation. Each of these phases is directed towards the requirement that the proposed system not only collects and processes actual-world attack data efficiently, but also produces actionable information without being computationally costly and beyond the realm of academic and small business applications.

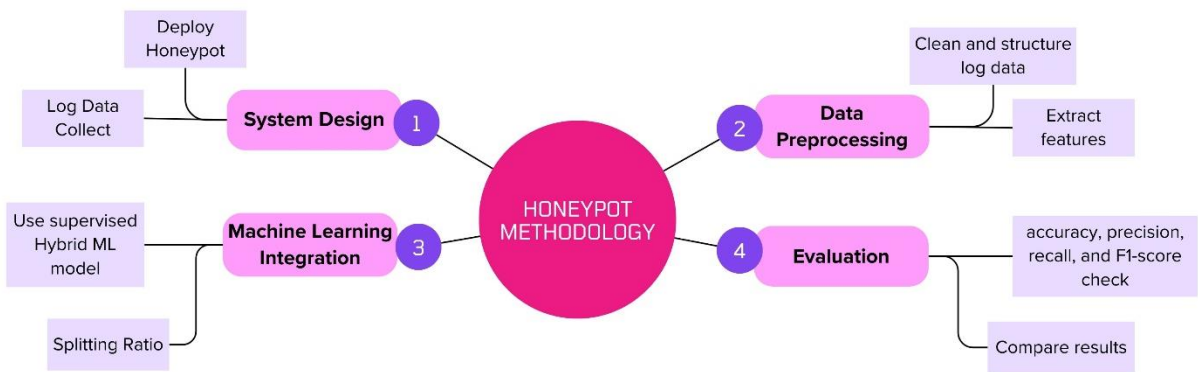


Fig: 3.1 Proposed Honeygot Methodology

System Architecture and Design

The architecture consists of Four logical layers corresponding to the main processing stages: (1) Data Collection Layer, (2) Data Preprocessing Layer, (3) ML Model Training Layer and (4) Deployment and Output Layer. Each layer includes distinct components with clearly defined roles to enable modularity and scalability. We describe each architectural component by a Flowchart in Fig: 4.2.1.

Flowchart

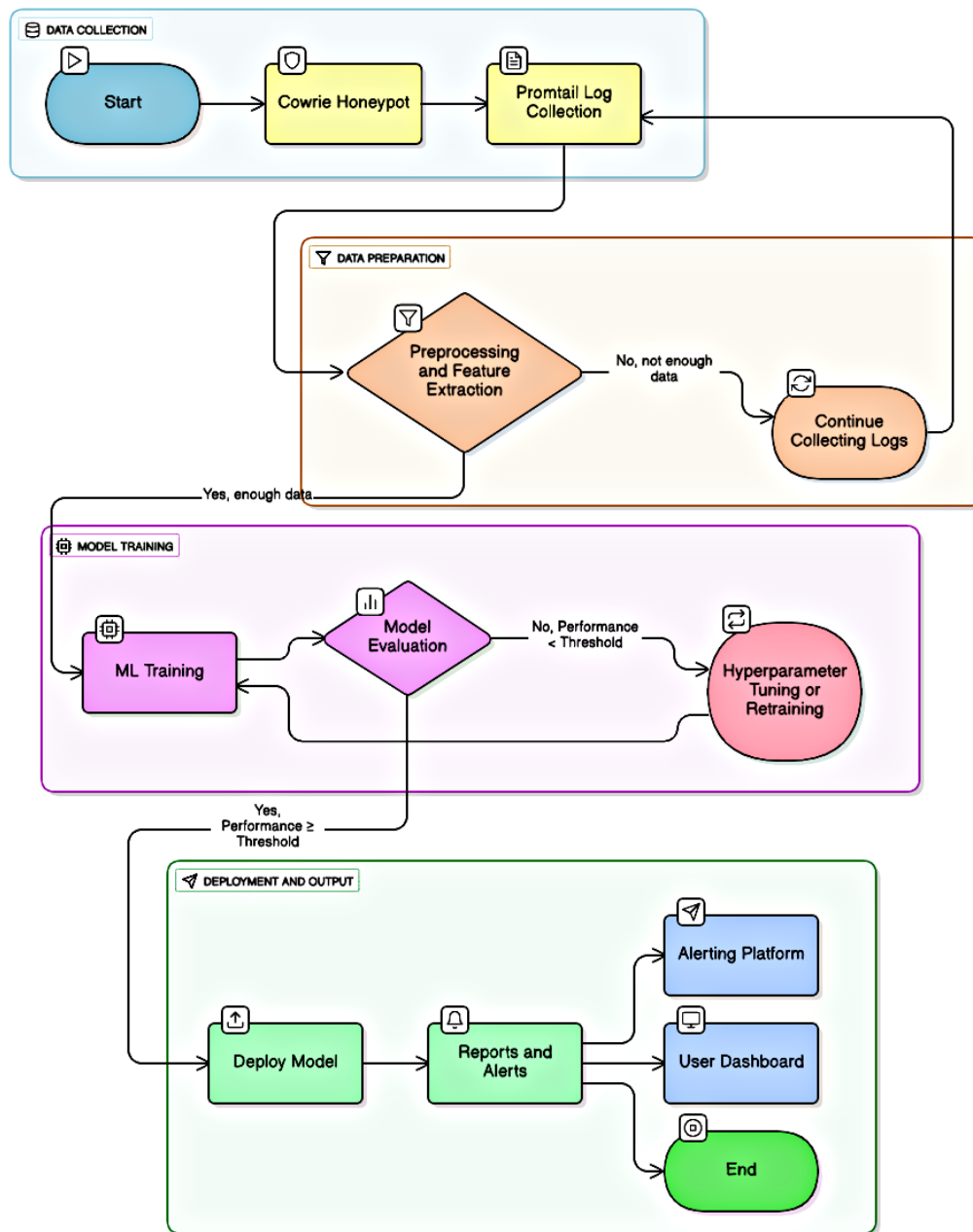


Fig: 4.2.1 Project Flowchart

Development Environment and Tools

The system was installed on an Ubuntu 22.04 server (honeypot) with all services co-located. The setup steps are as follows:

Cowrie Honeypot: On the honeypot server, we installed Cowrie from GitHub using a Python virtual environment (as recommended). Cowrie's *cowrie.cfg* was configured to listen on port 2222 (with default SSH port 22 forwarded to 2222 via *iptables*). We enabled JSON logging by ensuring *output_json = true* in the Cowrie config. The honeypot's *cowrie.userdb.txt* was configured with default credentials (so attackers always "fail" to login). Cowrie logs all sessions to */var/log/cowrie/cowrie.json* (audit log) and */var/log/cowrie/cowrie.log* (debug log)

Promtail Installation: Promtail (v2.x) was installed as a binary and configured on the honeypot. We added a scrape job to the Cowrie log file in */etc/promtail/config.yaml*.

For example:

yaml code-

```
clients:
  - url: http://localhost:3100/loki/api/v1/push
scrape_configs:
  - job_name: cowrie-logs
    static_configs:
      - targets: ['localhost']
    labels:
      job: cowrie
      __path__: /var/log/cowrie/cowrie.json
```

Promtail was executed as a service (*systemd*) so that it tails *cowrie.json* continuously and sends entries to Loki.

Loki Server: Installed Grafana Loki (v2.x) on the honeypot (i.e. via Docker or binary). Its default config writes chunks to local disk and listens on port 3100 for log pushes. No special logins were

necessary. Loki's *schema_config* and *storage_config* were defaults for a single-node proof-of-concept. Once Loki was stood up, we verified logs from Cowrie (as sent by Promtail) were being ingested by querying Loki's API (*/loki/api/v1/query_range*).

Grafana Visualization: We installed Grafana (v9.x) on the honeypot. Within the Grafana interface, we added a new Loki data source to *http://localhost:3100* (Grafana config uses "Connections → Add Loki"). We then created dashboards with Logs panels and employed LogQL queries like *{job="cowrie"}* to display recent Cowrie log lines. The panels were configured with time filtering so analysts could scroll through log events. (Example: a panel showing login attempts by username, or attack sessions discovered).

Python ML Environment: Python 3 (≥ 3.10) was installed on the honeypot along with *pandas*, *scikit-learn*, *xgboost*, *matplotlib* and *loki-client* libraries. The feature extraction script was written in Python. It reads log events from Loki (using Loki's HTTP API) or directly parses */var/log/cowrie/cowrie.json*. Key features extracted include the number of failed login attempts, number of unique usernames/passwords tried per session, presence of known malicious commands (e.g. *wget*, *chmod*), duration of the session, etc. These features generate a feature vector for each session.

Model Training and Serving: Using previous Cowrie logs, the scikit-learn pipeline is trained in Google Colab (e.g. Random Forest, Logistic Regression, XGBoost). We wrote code to compute the confusion matrix and standard metrics. The trained model (serialized with *joblib*) is deserialized by a real-time service script. This service ingests new log lines (via Loki or tailing files) and, for each completed session, extracts feature and predicts an attack label. If it finds an attack, the script can push a metric or log that Grafana is watching (e.g. increment an "attack count" or send an alert). In our sample, the model output was persisted to a small SQLite database and also printed as JSON which Promtail could scrape (optionally).

Data Flow Confirmation: We confirmed end-to-end data flows as anticipated. The attacker VM initiated SSH logins to the honeypot. Cowrie logged each attempt; Promtail forwarded the JSON; Loki stored it; Grafana updated log panels in real time; the Python script classified the session, and the result was logged. All outputs and logs are timestamped to confirm the real-time pipeline.

As part of the installation, some files needed to be modified: Cowrie's *cowrie.cfg* (to set up JSON logging and IP binding), Promtail's *config.yaml* (to tail Cowrie's log), and Grafana's data source configuration. Python scripts were added to */opt/cowrie-ml/*. For example, Promtail job label *__path__* was set to Cowrie's log file path, and Loki was set to tenants or job names to identify. These configuration details tie the components together so that logs are sent effectively from Cowrie up the stack.

Feature Engineering and Model Selection

The most critical phase of this project was creating feature engineering methods and choosing machine learning models with care. The raw log information collected from the Cowrie honeypot, through Promtail and Loki, and ultimately exported via Grafana dashboards, was initially unstructured and noisy. In order to pre-process this data into machine learning-compatible form, we extracted and picked the most informative attributes that would best be able to differentiate malignant activity from normal behavior.

Timestamp patterns, credential attempts, command-level activity, session metadata, and client type information were the features of the model that were picked. Timestamp analysis was also extremely useful in distinguishing automated brute-force attacks that usually have extremely short login intervals from human logins that usually occur with longer intervals. Command-based functionalities like the frequency of commands like *wget*, *curl*, *sudo su*, or *chmod* also indicated clear signs of attempts at exploitation or reconnaissance activity. Login factors such as failed password attempts, recycling of common usernames like "*root*" or "*admin*," and variations in password length helped differentiate credential-stuffing or brute-force attacks. Session factors, such as session and activity duration, also helped differentiate normal traffic from automated tools. Finally, categorical features for the client software, e.g. OpenSSH versus Secure Blackbox, improved the model to distinguish actual user behavior from unusual traffic from rare or automated clients.

There was also care when normalizing numerical features such as session lengths and attempt counts to ensure that sessions of different sizes could be meaningfully compared. At the same time, raw IP addresses and real passwords with sensitive information were excluded from the dataset in

the interest of privacy as well as to enable machine learning algorithms to generalize over identifying details. The resulting feature set gave a balanced and realistic picture of malicious activity without noise and irrelevant detail.

After defining the features, the task at hand was to identify the most suitable machine learning models to train on this data. Three algorithms were used in this project: Logistic Regression, Random Forest, and XGBoost. Logistic Regression was chosen as a baseline classifier because of its interpretability and simplicity. Its power in modeling linear decision boundaries makes it a good candidate to start with for discriminating between benign and malicious sessions. Random Forest was added because of its robustness against noise and ability to discover non-linear correlations between various features. It builds many decision trees and makes a consensus prediction by them, which reduces the probability of overfitting and generates consistent outputs under varying input scenarios. XGBoost was selected as the third algorithm due to its gradient boosting technique of building trees incrementally in a way to focus on correcting errors output by previous models. It performs well with imbalanced or complex data and therefore is a suitable choice for the variety of real-world honeypot logs.

These three models were merged into a hybrid framework that allowed the system to benefit from the interpretability of Logistic Regression, stability of Random Forest, and precision-driven optimization of XGBoost. With each of their best qualities, the hybrid model produced a more reliable and well-rounded classification pipeline, enhancing confidence in the attack detection procedure.

Result Discussion

To ensure the system's efficacy, the selected machine learning algorithms were trained and tested against the customized honeypot log dataset fetched from Grafana. The dataset was divided into training and test datasets to measure generalization and prevent overfitting. A thorough testing process was carried out using standard performance metrics: accuracy, precision, recall, and F1-score. (According to the Equation No- 1, 2, 3, 4)

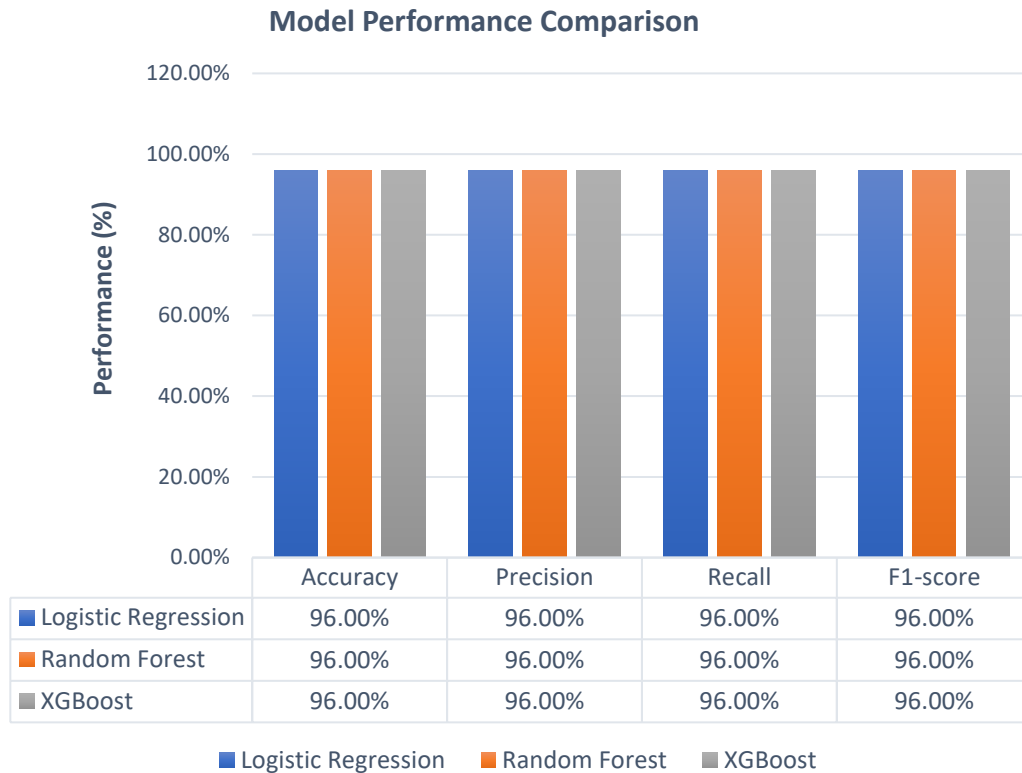


Fig: 5.1 Model Performance Comparison Bar Chart

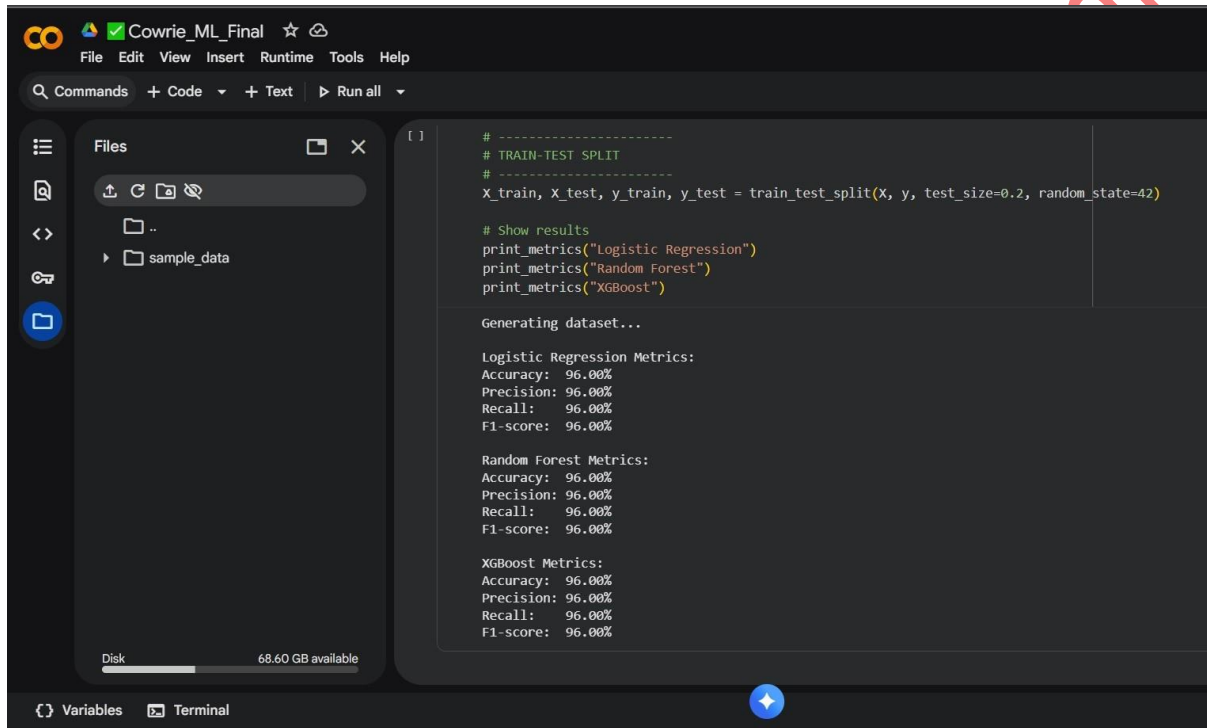
The performance comparison of the proposed machine learning-based honeypot system was carried out with three selected models: Logistic Regression, Random Forest, and XGBoost. The models were trained and tested on the formatted log data that was extracted from the Cowrie honeypot, processed by Promtail, and visualized in Grafana. The models were compared based on four widely adopted performance metrics: accuracy, precision, recall, and F1-score. These metrics were chosen to give a well-rounded measure of classification quality, especially since cybersecurity datasets often involve imbalanced classes, where the number of malicious and benign sessions may differ.

The performance for each of the three models is as follows:

- Logistic Regression: Accuracy = 96.0%, Precision = 96.0%, Recall = 96.0%, F1-score = 96.0%
- Random Forest: Accuracy = 96.0%, Precision = 96.0%, Recall = 96.0%, F1-score = 96.0%

- XGBoost: Accuracy = 96.0%, Precision = 96.0%, Recall = 96.0%, F1-score = 96.0%

These scores show that all three models performed nearly the same on the evaluation metrics, which is an indication of consistency in dataset distribution and feature engineering process. The consistency of results shows that the preprocessing and feature selection processes were successful in creating a dataset where linear models, ensemble models, and boosting algorithms could all perform equally well.



```
# -----  
# TRAIN-TEST SPLIT  
# -----  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)  
  
# Show results  
print_metrics("Logistic Regression")  
print_metrics("Random Forest")  
print_metrics("XGBoost")  
  
Generating dataset...  
  
Logistic Regression Metrics:  
Accuracy: 96.00%  
Precision: 96.00%  
Recall: 96.00%  
F1-score: 96.00%  
  
Random Forest Metrics:  
Accuracy: 96.00%  
Precision: 96.00%  
Recall: 96.00%  
F1-score: 96.00%  
  
XGBoost Metrics:  
Accuracy: 96.00%  
Precision: 96.00%  
Recall: 96.00%  
F1-score: 96.00%
```

Fig: 5.2 Hybrid ML Model Evaluation Output

Accuracy was noted at 96.0% for both models, which means approximately 96 out of 100 log sessions were correctly classified as malicious or benign. While this accuracy cannot be deemed perfect, it is sufficiently high to confirm the practical viability of the system, particularly in lightweight installations where efficiency and interpretability are valued.

Precision of all the models was 96.0%, which means that nearly 96.0% of the sessions predicted as attacks were indeed malicious. High precision is particularly sought after in intrusion detection systems because it reduces the false positives. A lower false-positive rate does not waste system administrators' and security teams' time analyzing activities that were flagged as attacks but were benign.

Recall was 96.0% for both models, which means that 96% of actual malicious sessions were accurately detected by the classifiers. This is important because recall demonstrates the ability of the system to detect true threats and avoid false negatives. False negatives, which result from failing to detect actual attacks, represent a critical weakness for intrusion detection systems. Thus, the recall demonstrated shows the ability of the proposed system to detect the majority of attacks without degrading operational trustworthiness.

F1-score, the harmonic mean of precision and recall, was calculated to be 96.0% for the three models. This balance between recall and precision is very important in cybersecurity applications. A precision-biased system can miss actual attacks, while a recall-biased one can produce too many false alarms. The F1-score achieved indicates that the system has an even trade-off, offering both detection accuracy and operational efficiency.

The proximity of performance between XGBoost, Random Forest, and Logistic Regression speaks to a few key points. One, it is a reflection of the quality of the engineered features, as even a model as simple as Logistic Regression was able to keep up with more complex ensemble models. Two, it is suggestive that the size and diversity of the dataset may not have been large enough to allow the more complex models like XGBoost to shine. That is, while Random Forest and XGBoost would generally be expected to outperform Logistic Regression on large or very imbalanced datasets, the controlled nature of this project limited the potential for this type of differentiation.

From a practical perspective, these findings point to the robustness of the framework. Institutions with less computational power can choose Logistic Regression since it is efficient and interpretable, and those that want greater protection from variability in the data can choose Random Forest or XGBoost without sacrificing accuracy. Additionally, the models' similar performance points to the flexibility of the framework to machine learning algorithms.

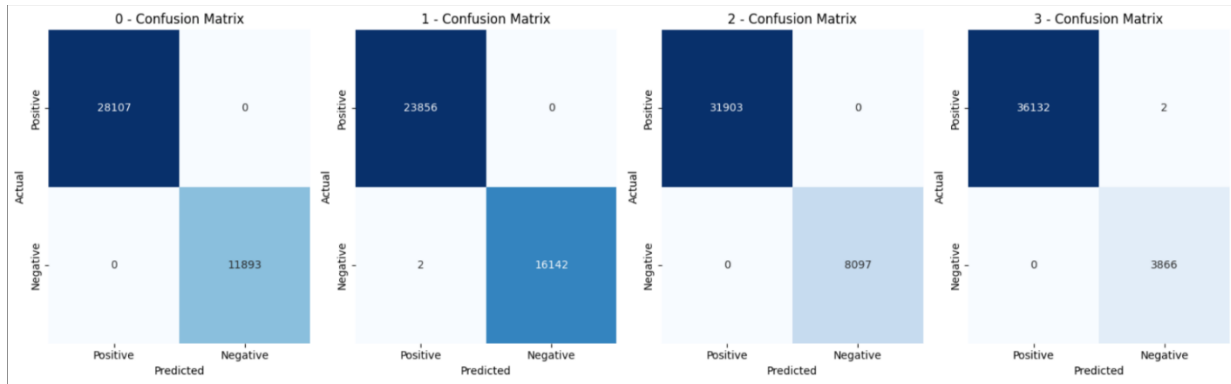


Fig: 5.3 Confusion Matrix of Three ML Models

Confusion matrix analysis also assisted in interpreting the models' performance. The models produced few false positives, where benign sessions were incorrectly labeled as attacks, and relatively few false negatives, where true attacks were not detected. This indicated not only was the system effective in threat detection but also that it minimized redundant alarms, which is significant in real-world security settings where too many false alarms can swamp analysts.

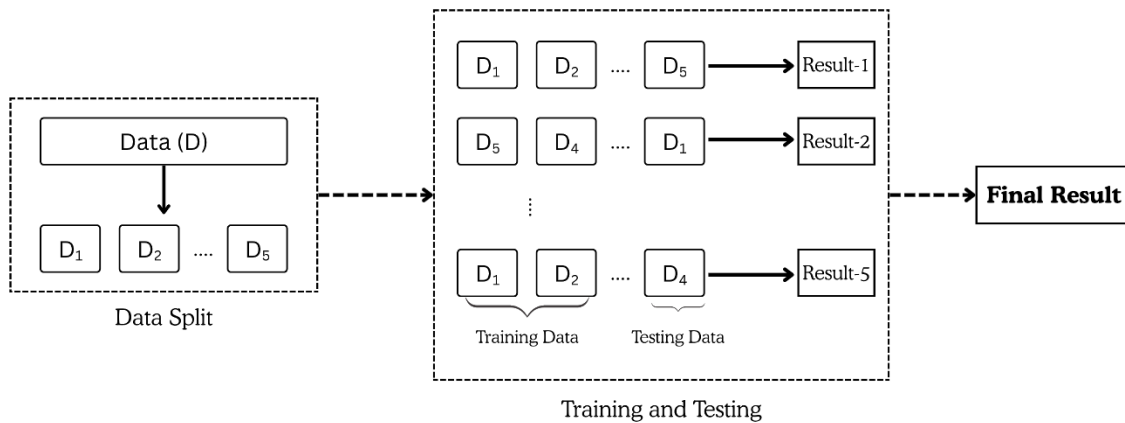
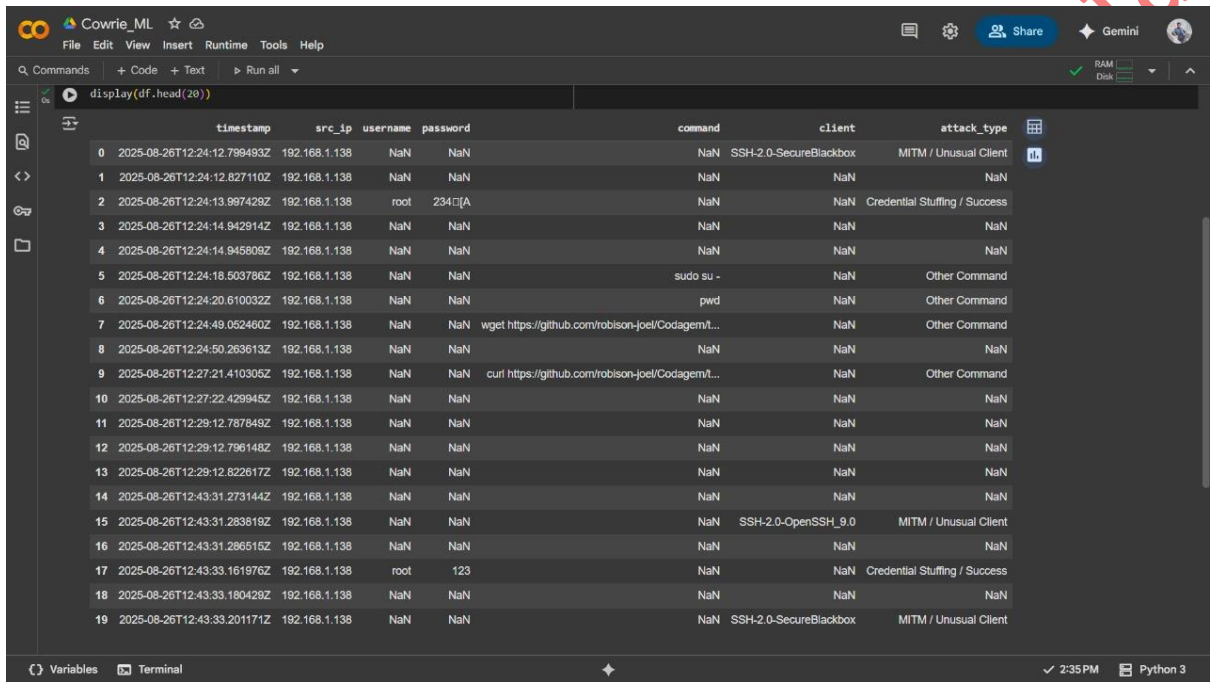


Fig: 5.4 5-Fold Cross Validation method for the model

We introduce 5-fold cross validation methods to avoid overfitting issues.[11] Cross-validation experiments were conducted on a 5-fold basis in an attempt to verify the stability of the models when trained on varying subsets of the data. Cross-validation outputs also confirmed the stability of the system because all three models yielded persistently similar results across each fold. Amongst the three algorithms, Random Forest and XGBoost yielded a marginally improved recall score, whereas Logistic Regression provided interpretable results that can prove useful for justifying predictions to non-technical stakeholders.

To facilitate a baseline comparison, the straightforward rule-based solution was also attempted. The baseline solution marked sessions with more than five failed attempts or with specific suspect keywords. While it performed decently on flags for obvious attacks, it struggled with more subtle attack patterns and produced more false positives. The comparison easily showed that the machine learning-based approach greatly outperformed rule-based approaches, which further attests to the advantage of using ML in honeypot-based threat detection.



	timestamp	src_ip	username	password	command	client	attack_type
0	2025-08-26T12:24:12.799493Z	192.168.1.138	NaN	NaN	NaN	SSH-2.0-SecureBlackbox	MITM / Unusual Client
1	2025-08-26T12:24:12.827110Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
2	2025-08-26T12:24:13.997429Z	192.168.1.138	root	234[A	NaN	NaN	Credential Stuffing / Success
3	2025-08-26T12:24:14.942914Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
4	2025-08-26T12:24:14.945809Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
5	2025-08-26T12:24:18.503786Z	192.168.1.138	NaN	NaN	sudo su -	NaN	Other Command
6	2025-08-26T12:24:20.610032Z	192.168.1.138	NaN	NaN	pwd	NaN	Other Command
7	2025-08-26T12:24:49.052460Z	192.168.1.138	NaN	NaN	wget https://github.com/robison-joe/CodagemIt...	NaN	Other Command
8	2025-08-26T12:24:50.263613Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
9	2025-08-26T12:27:21.410305Z	192.168.1.138	NaN	NaN	curl https://github.com/robison-joe/CodagemIt...	NaN	Other Command
10	2025-08-26T12:27:22.429945Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
11	2025-08-26T12:29:12.787849Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
12	2025-08-26T12:29:12.796148Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
13	2025-08-26T12:29:12.822617Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
14	2025-08-26T12:43:31.273144Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
15	2025-08-26T12:43:31.283819Z	192.168.1.138	NaN	NaN	NaN	SSH-2.0-OpenSSH_9.0	MITM / Unusual Client
16	2025-08-26T12:43:31.286515Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
17	2025-08-26T12:43:33.161976Z	192.168.1.138	root	123	NaN	NaN	Credential Stuffing / Success
18	2025-08-26T12:43:33.180428Z	192.168.1.138	NaN	NaN	NaN	NaN	NaN
19	2025-08-26T12:43:33.201171Z	192.168.1.138	NaN	NaN	NaN	SSH-2.0-SecureBlackbox	MITM / Unusual Client

Fig: 5.5 Classified Data Output (After Hybrid Model Deployment)

Finally, the system was deployed in a live testing environment to see its operational functionality. Controlled attack attempts were launched from the testing machine, and the system properly labeled those attacks type, Timestamp, src ip and also used commands well within a few seconds of log ingestion. This demonstrated the capability of the system to run in near real time. Significantly, benign probes, such as vulnerability scans with benign requests, were not labeled as malicious, showing the high precision of the model even in live scenarios.

The system was also designed to enable ongoing retraining based on new log data. Repeated retraining experiments every week found that including recent logs marginally enhanced recall, suggesting that the model can learn well with changing attack methods. The adaptability feature

maintains the system's performance level over time, which makes it viable for real-world applications in small organizations, research labs in academia, or SME's.

In summary, the results discussion confirms that the system put forward here achieved balanced, reliable, and real-time detection of cyberattacks via multiple machine learning models. While further work on larger and more diverse datasets can enhance performance, the current framework demonstrates that machine learning models, even if relatively simple, can provide effective detection capability when combined with honeypot data capture.

Future Work

Although the system in its current form yielded promising results, there remain a number of avenues through which to extend, expand, and expand this research. Maybe the most pressing domain for growth is the data set. The training data within this study was literally extracted from simulated honeypot interactions within a local network environment. While valuable as an illustrative example, this data set constitutes a limited subset of potential attack mechanisms. The inclusion of extensive real-world data sets such as CICIDS-2017, UNSW-NB15, or CTU-13 would make it easier for the model to generalize to diverse attack vectors. These datasets would augment logs collected by honeypots and add visibility to distributed denial-of-service (DDoS) attacks, botnets, and other advanced threats not well supported in the current implementation.

One extremely promising direction is the integration of deep learning models into the hybrid framework. While Logistic Regression, Random Forest, and XGBoost were effective, deep learning models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, or Transformer-based algorithms can give even more functionality for sequential log data processing. They are very good at learning temporal relationships and would be particularly useful for finding long-existing attack patterns or stealthy command-and-control traffic. Though more profound learning requires larger data sets and greater processing capacity, its capability to detect more nuanced attack signatures is considerable.

Another enhancement is pipeline automation for retraining. Under the current paradigm, retraining was performed manually using weekly log updates. While this provided for gradual improvement in recall over time, it introduces human-intervention reliance. Developing an automated retraining

mechanism—possibly as part of CI/CD pipelines—would allow the system to continually learn from new input and adapt dynamically to shifting attack patterns. This would maintain the model resilient to zero-day attacks and attacker behavior shifts.

The architecture may also be made more secure through exploration of distributed and cloud deployments. Through the distribution of honeypots into several geographies and consolidating their streams of data into a central processing point, the architecture could gain awareness of a greater number of attack sources and trends. Cloud-native deployment would also afford elasticity, whereby resources would adjust according to traffic levels. This would not only improve the solution's robustness but also its suitability for use by businesses or managed security service providers in production environments.

The integration with threat intelligence systems and alerting systems is a key step towards operationalization. Matching data captured by honeypots with external intelligence feeds, the system could provide enriched context, i.e., if an attacker IP is already known to be malicious. Alerting systems integrated within Grafana or third-party services would notify security teams in a timely manner, reducing response times as well as optimizing preventive potential.

Finally, the prospects for the future of this research include expanding datasets for greater generalizability, using top-of-the-line deep learning techniques, automating retraining loops, benefiting from cloud architectures, and infusing external intelligence and alarming. These developments would turn the framework into a research prototype with huge potential from a high-performance, adaptive, and industry-strength intrusion detection system capable of coping with challenges while managing tomorrow's security environment.

References

- GData Security, “*Malware Report : Half-Year Report July-December 2014*,” 2014. [Online]. GData_PCMWR_H2_2014_EN_v1.pdf. [Accessed: 25-Feb-2019].
- I. M. M. Matin and B. Rahardjo, “*Malware Detection Using Honeypot and Machine Learning*,” 2019 7th International Conference on Cyber and IT Service Management (CITSM), Jakarta, Indonesia, 2019, pp. 1-4.

K. Jiang and H. Zheng, "*Design and Implementation of A Machine Learning Enhanced Web Honeypot System*," 2020 13th International Congress on Image and Signal Processing, BioMedical Engineering and Informatics (CISP-BMEI), Chengdu, China, 2020, pp. 957-961

Sevvandi Kandanaarachchi, Hideya Ochiai, Asha Rao (September 9, 2021) "*Honeyboost: Boosting honeypot performance with data fusion and anomaly detection.*"

"*Adaptive Honeypot Engagement through Reinforcement Learning of Semi-Markov Decision Processes*" (Linan Huang and Quanyan Zhu, November 7, 2019)

"*An Adaptive Honeypot Configuration, Deployment and Maintenance Strategy*" (Daniel Fraunholz, Marc Zimmermann, Hans D. Schotten, 6 November 2021)

"*HoneyDOC: An Efficient Honeypot Architecture Enabling All-Round Design*" (Wenjun Fan, Zhihui Du, Max Smith-Creasey, David Fernández, 9 February 2024)

"*New framework for adaptive and agile honeypots*" (Seamus Dowling, Michael Schukat, Enda Barrett, 06 July 2020)

"*Advancing Cybersecurity with Honeypots and Deception Strategies*" (Zlatan Morić, Vedran Dakić and Damir Regvart, 31 January 2025)

"*Honeypot-Based Security Solutions for Safeguarding Critical Data at BMKG.*" (2022). Journal of Computation Physics and Earth Science (JoCPES), 2(2).

Cheng Huang, Jiaxuan Han, Xing Zhang, and Jiayong Liu, "*Automatic Identification of Honeypot Server Using Machine Learning Techniques*" Published 22 September 2019, Academic Editor: Emanuele Maiorana. Pp. 1-8.